The Two-Pointer Technique & Floyd's Tortoise and Hare Algorithm

A complete guide to slow/fast pointers: concepts, proofs, code, and applications

This guide explains the **two-pointer technique** — a fundamental pattern in algorithm design — with special focus on its most famous variant, **Floyd's Cycle Detection Algorithm**, popularly known as the **Tortoise and Hare algorithm**. It covers the intuition, step-by-step mechanics, mathematical proof of correctness, complexity analysis, real interview problems, and working Python code.

Contents

1. What Is the Two-Pointer Technique?

The **two-pointer technique** is a strategy where two index variables ('pointers') traverse a data structure — usually an array, string, or linked list — often at different speeds or from different directions, instead of using nested loops. It converts many $O(n^2)$ brute-force solutions into $O(n)$ solutions by avoiding redundant re-scanning of already-seen elements.

Why use two pointers?

- Reduces time complexity — typically from $O(n^2)$ to $O(n)$.
- Uses $O(1)$ extra space in most cases — no auxiliary arrays or hash maps needed.
- Works naturally on sorted arrays, linked lists, and strings.
- Elegant and easy to reason about once the pattern is recognized.

2. Common Two-Pointer Patterns

a) Opposite-direction pointers

One pointer starts at the beginning (`left`) and the other at the end (`right`) of a sorted array; they move toward each other. Used in problems like *Two Sum (sorted array)*, *container with most water*, and *reversing an array in place*.

b) Same-direction pointers (fast/slow, sliding window)

Both pointers start at the beginning but move at different rates or maintain a window between them. Used in *removing duplicates*, *sliding window sums*, and — most famously — **cycle detection in linked lists**.

c) Slow/Fast pointers (Floyd's algorithm family)

A special case of same-direction pointers where the **slow pointer** moves one step at a time and the **fast pointer** moves two steps at a time. This speed difference is the key idea behind Floyd's Tortoise and Hare algorithm, discussed in depth below.

---

3. Floyd's Tortoise and Hare Algorithm — Introduction

Floyd's Cycle Detection Algorithm, invented by **Robert W. Floyd**, answers a simple but important question: **given a linked list (or any sequence generated by repeatedly applying a function), does it contain a cycle — and if so, where does the cycle begin?**

The algorithm is named after Aesop's fable of the tortoise and the hare: a slow-moving 'tortoise' pointer and a fast-moving 'hare' pointer traverse the same list. If the list loops back on itself, the faster hare will eventually 'lap' the tortoise and the two will meet at the same node — proving a cycle exists. If the list has no cycle, the hare simply reaches the end (`null`) first.

> This is also called the **slow/fast pointer technique**, since the tortoise advances one node per step while the hare advances two nodes per step.

4. How It Works — Step by Step

- **Step 1:** Initialize two pointers, `slow` and `fast`, both pointing to the head of the list.
- **Step 2:** Move `slow` forward by 1 node, and `fast` forward by 2 nodes, on every iteration.
- **Step 3:** If `fast` or `fast.next` becomes `null`, the list has **no cycle** — stop and return false.
- **Step 4:** If at any point `slow == fast` (they point to the same node), a **cycle exists** — stop and return true.
- **Step 5 (optional):** To find where the cycle *begins*, reset one pointer to the head and move both pointers one step at a time; the node where they meet again is the cycle's start (proved in Section 5).

Visualizing the chase

Imagine a list: 1 → 2 → 3 → 4 → 5 → 6 → back to 3 (a cycle starting at node 3). The table below shows the position of each pointer after every step.

Step	slow (×1)	fast (×2)
0	1	1
1	2	3
2	3	5
3	4	4
4	5	6
5	6	5
6	3	4
7	4	6
8	5	5
9	6	4 ← meet! slow == fast

Node values shown are list positions; once inside the loop, the fast pointer wraps around faster and eventually lands on the same node as the slow pointer — confirming a cycle.

5. Why It Works — Mathematical Proof

Let the distance from the head of the list to the start of the cycle be μ (mu), and let the cycle length be λ (lambda).

Claim: the pointers must eventually meet inside the cycle.

Once both pointers have entered the cycle, think of their positions modulo λ. On every iteration, the gap between fast and slow (fast – slow, measured going forward around the cycle) increases by exactly 1 node, because fast gains 2 and slow gains 1 per step. Since the cycle has finite length λ, the gap must eventually become an exact multiple of λ — i.e. 0 mod λ — meaning both pointers occupy the same node. This must happen within at most λ iterations after both are inside the loop.

Claim: finding the cycle start using μ and λ.

When slow and fast first meet, slow has traveled `μ + k` steps for some `0 ≤ k < λ`, and fast has traveled exactly twice as far: `2(μ + k)`. Since fast has also gone around the loop some integer number of times more than slow, the extra distance `2(μ+k) – (μ+k) = μ+k` must be a multiple of λ. So `μ + k = n·λ` for some integer n, which gives `μ = n·λ – k`.

This means: if you move a pointer from the **head** forward by μ steps, and separately move the meeting-point pointer forward by μ steps (which is the same as moving n·λ – k steps — a whole number of loops minus k), **both land on the same node**: the start of the cycle. That is exactly why resetting one pointer to the head and advancing both one step at a time (Step 5 above) correctly finds the cycle's starting node.

> In short: distance from head to cycle start (μ) equals distance from the meeting point to the cycle start, measured going forward around the loop. This elegant symmetry is the heart of Floyd's algorithm.

6. Finding the Start of the Cycle

- Phase 1: Run slow/fast pointers until they meet (confirms a cycle exists).
- Phase 2: Reset `slow` to the head; keep `fast` at the meeting point.
- Phase 3: Move both pointers one step at a time. The node where they meet again is the first node of the cycle.

7. JavaScript Implementation

Node definition:

```
class ListNode {
constructor(val, next = null) {
this.val = val;
this.next = next;
}
}
```

Basic cycle detection (returns true/false):

```
function hasCycle(head) {
let slow = head;
let fast = head;

while (fast !== null && fast.next !== null) {
slow = slow.next; // tortoise: 1 step
fast = fast.next.next; // hare: 2 steps

if (slow === fast) { // they met -> cycle found
return true;
}
}

return false; // fast reached the end -> no cycle
}
```

Finding the start of the cycle:

```
function detectCycleStart(head) {
let slow = head;
let fast = head;
let cycleFound = false;

// Phase 1: determine if a cycle exists
while (fast !== null && fast.next !== null) {
slow = slow.next;
fast = fast.next.next;
if (slow === fast) {
cycleFound = true;
break;
}
}

if (!cycleFound) return null; // no cycle

// Phase 2: find the entry point of the cycle
slow = head;
while (slow !== fast) {
slow = slow.next;
fast = fast.next;
}

return slow; // this node is the start of the cycle
}
```

Bonus: finding the middle of a linked list (same pattern, no cycle):

```
function findMiddle(head) {
let slow = head;
let fast = head;

while (fast !== null && fast.next !== null) {
slow = slow.next;
fast = fast.next.next;
}

return slow; // slow is now at the middle node
}
```

Happy Number (LeetCode 202) — cycle detection on a number sequence:

A number is 'happy' if repeatedly replacing it with the sum of the squares of its digits eventually reaches 1. If it never reaches 1, it loops forever in a cycle — which the tortoise and hare can detect without needing a hash set.

```
function sumOfSquares(n) {
let total = 0;
while (n > 0) {
const digit = n % 10;
total += digit * digit;
n = Math.floor(n / 10);
}
return total;
}

function isHappy(n) {
let slow = n;
let fast = n;

do {
slow = sumOfSquares(slow); // 1 step
fast = sumOfSquares(sumOfSquares(fast)); // 2 steps
} while (slow !== fast);

return slow === 1; // if they met at 1, n is happy
}
```

Find the Duplicate Number (LeetCode 287) — array as an implicit linked list:

Given an array of n+1 integers where each value is between 1 and n, there must be at least one duplicate. Treating each value as a 'pointer' to the index it names turns the array into a linked list with a cycle — and the duplicate value is exactly the cycle's entry point.

```
function findDuplicate(nums) {
let slow = nums[0];
let fast = nums[0];

// Phase 1: find the meeting point inside the cycle
do {
slow = nums[slow];
fast = nums[nums[fast]];
} while (slow !== fast);

// Phase 2: find the entrance to the cycle (the duplicate)
slow = nums[0];
while (slow !== fast) {
slow = nums[slow];
fast = nums[fast];
}

return slow; // the duplicate number
}
```

Both problems above reuse the exact same slow/fast pattern as linked-list cycle detection — proof that Floyd's algorithm generalizes to *any* sequence defined by repeatedly applying a function, not just literal linked lists.

8. Dry Run Example

Consider the list: 1 → 2 → 3 → 4 → 5 → 6 → 3 (node 6 points back to node 3, forming a cycle of length λ = 4, with μ = 2 nodes before the cycle).

Iteration	slow	fast
Start	1	1
1	2	3
2	3	5
3	4	4 (fast wrapped: 5→6→3→4)
4	5	6
5	6	5 ← MEET

They meet at node 5. Now reset slow to head (node 1) and move both one step at a time: 1→2, 5→6 ... then 2→3, 6→3 → they meet at node 3, correctly identified as the cycle start (μ = 2 steps from head, matching the proof in Section 5).

9. Complexity Analysis

Operation	Time Complexity	Space Complexity
Cycle detection (has_cycle)	O(n)	O(1)
Finding cycle start	O(n)	O(1)
Finding the middle element	O(n)	O(1)
Naive approach (hash set of visited nodes)	O(n)	O(n)

Floyd's algorithm is preferred over the hash-set approach specifically because it achieves O(1) space — no extra memory is needed to remember visited nodes.

10. Real-World Applications & Interview Problems

- **Linked List Cycle Detection** — LeetCode 141 (Linked List Cycle).
- **Finding Cycle Start** — LeetCode 142 (Linked List Cycle II).
- **Finding the Middle of a Linked List** — LeetCode 876.
- **Happy Number problem** — detecting cycles in a sequence generated by summing squares of digits (LeetCode 202).

- **Duplicate Number Detection** — LeetCode 287 (Find the Duplicate Number), which cleverly maps an array to a linked-list-like structure using indices as 'next' pointers.
- **Detecting infinite loops** in functional/iterative processes, such as pseudo-random number generators or hash-chain traversal.
- **Cryptography** — Floyd's algorithm (and its refinement, Brent's algorithm) is used in Pollard's rho algorithm for integer factorization.

11. Summary Cheat Sheet

Concept	Key Idea
Two-pointer technique	Use two indices/pointers instead of nested loops to cut time complexity.
Slow pointer (tortoise)	Moves 1 step per iteration.
Fast pointer (hare)	Moves 2 steps per iteration.
Cycle exists if...	slow and fast pointers ever point to the same node.
No cycle if...	fast pointer (or fast.next) reaches null/end of list.
Finding cycle start	Reset slow to head; move both 1 step at a time until they meet again.
Time complexity	O(n)
Space complexity	O(1)

Master this pattern once, and you'll recognize it everywhere: any time a problem involves 'find the middle', 'detect a loop', or 'two things moving at different speeds', reach for the tortoise and the hare.